CHAPTER 12

TCP Segment & Connection Management

Header · sequence and ACK numbers · 3-way handshake · 4-way close · states · RTO · sequence wrap-around

QUICK TRICK

Handshake: **SYN(x) → SYN+ACK(y, ack x+1) → ACK(x+1, ack y+1)**. **SYN and FIN each use one sequence number**; a pure ACK uses none. ACK number = **next byte expected**. MSS = MTU − 40 = **1460** on Ethernet.

12.1 What TCP gives you

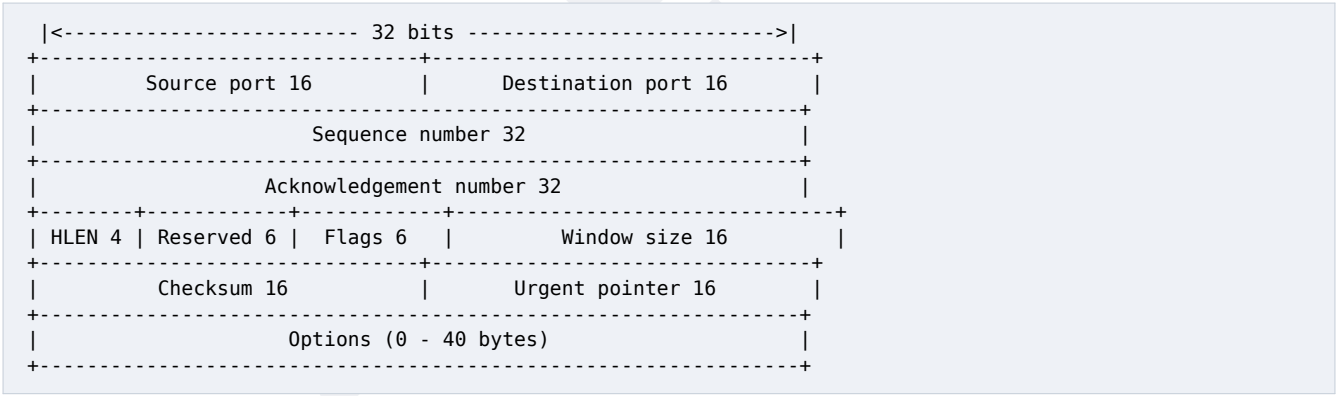
● **LEVEL 1 · BEGINNER**

TCP (Transmission Control Protocol) sits above IP and gives applications a **reliable, ordered byte stream** over an unreliable network. It is **connection-oriented**, **full duplex** (both ways at once), and has **flow control** and **congestion control**. It numbers every **byte**, not every segment.

Analogy. A phone call with receipts. You first ring and agree to talk (handshake). Every few words the listener says "got everything up to here" (ACK). Then you both say goodbye (close).

12.2 The TCP header

● **LEVEL 2 · EXAM**



Field	Facts
Ports	16 bits each. Connection = (src IP, src port, dst IP, dst port).
Sequence number	Number of the **first data byte** in this segment (32 bits).
ACK number	Next byte the receiver expects (cumulative acknowledgement).
HLEN (data offset)	Header length in 4-byte words. Min 5 (20 B), max 15 (60 B).
Flags	URG, **ACK**, PSH, RST, **SYN**, **FIN**.
Window size	16 bits: receiver's advertised window (max 65535 B; window scaling extends it).
Checksum	Covers header + data + pseudo-header (as in Chapter 4).

12.3 Sequence and ACK numbers

● **LEVEL 2 · EXAM**

- Sequence number = byte number of the first data byte. ACK = **sequence + data length** of the last in-order segment (the next byte wanted).
- **SYN and FIN consume 1 sequence number** each. A segment with no data and only ACK consumes 0.

- Example: segment seq = 2000 carries 300 B → receiver ACKs **2300**. Segment with seq = 5000, 1000 B → ACK 6000.

12.4 Connection setup: 3-way handshake

● LEVEL 2 · EXAM

```
Client                                  Server
  |  -------- SYN, seq = x ---------->   |   (client: SYN_SENT;   server: LISTEN)
  |  <---- SYN+ACK, seq = y, ack = x+1 -  |   (server: SYN_RCVD)
  |  -------- ACK, ack = y+1 ---------->  |   (both: ESTABLISHED)   data may follow
```

First data byte sent by the client has sequence number **x + 1** (the SYN used x). Why three steps? Both sides must learn the other's initial sequence number and confirm it.

12.5 Connection close: 4-way

● LEVEL 2 · EXAM

```
A (active close)                        B
  |  -------- FIN, seq = u ---------->   |   A: FIN_WAIT_1
  |  <------- ACK, ack = u+1 ----------   |   A: FIN_WAIT_2     B: CLOSE_WAIT
  |  <------- FIN, seq = v ------------   |   B: LAST_ACK
  |  -------- ACK, ack = v+1 --------->   |   A: TIME_WAIT (2 x MSL) -> CLOSED   B: CLOSED
```

- Each direction closes separately (**half-close**). FIN consumes one sequence number.
- **TIME_WAIT = 2 × MSL**: lets the last ACK be resent if lost, and lets old duplicates from this connection die out.
- Segments for full life of a connection: 3 (open) + 4 (close) = **7** (3 if the middle ACK and FIN are combined).
- **RST** aborts a connection at once.

State	Meaning
LISTEN	Server waiting for a SYN
SYN_SENT / SYN_RCVD	Handshake in progress
ESTABLISHED	Data transfer
FIN_WAIT_1 / FIN_WAIT_2	Active closer sent FIN / got its ACK
CLOSE_WAIT / LAST_ACK	Passive side got FIN / sent its own FIN
TIME_WAIT	Wait 2·MSL, then CLOSED

12.6 Numbers you must know

● LEVEL 3 · GATE

- **MSS** = MTU − 20 (IP) − 20 (TCP) = **1460 B** for Ethernet (no options).
- **Sequence wrap-around:** 2^{32} bytes. Time to wrap = 2^{32} × 8 / rate. To avoid reuse within one MSL: **rate ≤ 2^{32} × 8 / MSL**. With MSL = 60 s this is ≈ **572.7 Mbps**.
- **RTT estimate and timeout (RFC 6298 / Jacobson):** EstimatedRTT = (1 − α)·EstimatedRTT + α·Sample (α = 1/8). DevRTT = (1 − β)·DevRTT + β·|Sample − EstimatedRTT| (β = 1/4). **RTO = EstimatedRTT + 4·DevRTT**.
- **Karn's rule:** never take an RTT sample from a retransmitted segment.

12.7 Common mistakes

COMMON MISTAKE

- Giving the ACK as the sequence number of the segment received (it is the **next** byte wanted).
- Forgetting that SYN and FIN consume a sequence number (so data starts at x + 1).
- Counting segments instead of bytes when computing sequence numbers.
- Using MSS = MTU (subtract the 40 B of IP and TCP headers).

QUICK TRICK

Memory line: "SYN and FIN cost one number; ACK says what comes next."

GATE QUICK CHECK — Chapter 12

```
Header 20..60 B      ACK = seq + bytes      SYN, FIN = 1 seq number each      pure ACK = 0
Open : SYN, SYN+ACK, ACK      Close : FIN, ACK, FIN, ACK      TIME_WAIT = 2*MSL
MSS = MTU - 40       wrap rate <= 2^32*8/MSL      RTO = EstRTT + 4*DevRTT
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Minimum and maximum TCP header size.

Q2. A segment with seq = 2000 carries 300 B. What ACK number is returned?

Q3. A SYN with seq = 100 is received. What ACK number is in the SYN+ACK?

Q4. How many segments for a full open and close (no combining)?

● LEVEL 2 · EXAM

Q5. MSS for an Ethernet link with MTU 1500 and no options. How many segments for 10,000 B and what is the last one's size?

Q6. A client sends SYN with ISN 1000, then three 100 B data segments. Give the data sequence numbers and the final ACK from the server.

Q7. A FIN with seq = 500 is received. What is the ACK number?

Q8. Maximum data rate so that the 32-bit sequence number does not wrap within MSL = 60 s?

● LEVEL 3 · GATE

Q9. EstimatedRTT = 100 ms, DevRTT = 10 ms. New sample = 140 ms. Find the updated EstimatedRTT, DevRTT and RTO ($\alpha = 1/8$, $\beta = 1/4$).

Q10. Why does the active closer wait in TIME_WAIT for 2 × MSL?

Q11. Why can a pure ACK segment carry the same sequence number repeatedly?

Answers

Q	Solution
1	**20 B** and **60 B** (HLEN 5 to 15 words).
2	**2300**.
3	**101**.
4	3 + 4 = **7**.
5	MSS = **1460**. ⌈10000/1460⌉ = **7** segments; the last carries 10000 − 6×1460 = **1240 B**.
6	Data seq numbers: **1001, 1101, 1201**. Final ACK = **1301**.
7	**501**.
8	2^{32} × 8 / 60 ≈ **572.7 Mbps**.
9	Est = 0.875×100 + 0.125×140 = **105 ms**. Dev (using the old estimate, as RFC 6298) = 0.75×10 + 0.25×40 = **17.5 ms**. RTO = 105 + 4×17.5 = **175 ms**. (If a question uses the new estimate, |140 − 105| = 35 gives Dev = 16.25 and RTO = 170 ms; follow the order the question states.)
10	To retransmit the last ACK if it was lost, and to let old duplicate segments of this connection die out before the same ports are reused.
11	It carries no data and consumes **no** sequence number, so the next sequence number does not advance.

Next: Chapter 13 — TCP Flow Control and Congestion Control.